Module 1 Quiz

Review Learning Objectives

Assignment details

Due
Dec 10, 11:59 PM IST

Attempts
Unlimited

Submitted
Dec 2, 1:17 AM IST

Your grade
To pass you need at least 80%. We keep your latest score.

100%

Like Dislike Report an issue

Your grade: 100%

Your latest: **100%** • Your highest: **100%** • To pass you need at least 80%. We keep your latest score.

Next Item

Try again

View submission See feedback

1. In the context of GPT, what is a *Transformer*? **1 / 1 point**

- Alien robots
- An electrical component that transfers electrical energy between circuits
- A neural network architecture
- A software component that converts a string into tokens

Correct
Correct! Transformers are a specialized type of neural network architecture that GPT uses to model language.

2. What type of language model predicts probabilities in the form of P("GPT"|"I","am","learning","about")? **1 / 1 point**

- Pretrained Language Models
- Bidirectional Language Model
- Causal Language Models
- Masked Language Models

Correct
Correct! Causal language models predict the final word in a sequence

3. Does the probability provided by a language model depend on a reference corpus? **1 / 1 point**

- Yes, there is no single "correct" answer to the probability of specific text because language differs among people, culture, context, and time.
- No, we can build a single universal language model that covers every language.
- No, we can use a language model like GPT instead of using a corpus.
- Yes, we need to have enough text to have seen every possible sentence to measure its probability.

Correct
Correct! Due to the variety of language, we need to measure the probability of text with respect to a set of reference text.

4. Which of the following is a key advantage of using a lower 'n' value in an n-gram language model over a higher 'n' value? **1 / 1 point**

- Lower 'n' values result in more unique n-grams.
- Lower 'n' values improve the model's performance on unseen data.
- Lower 'n' values increase the model's complexity.
- Lower 'n' values result in less data sparsity.

Correct
Good job! Lower 'n' values do result in less data sparsity. This is because there are fewer unique n-grams, which makes it more likely that an n-gram in the test data was also in the training data.

5. Which of the following is NOT a type of n-gram? **1 / 1 point**

- Trigram
- Unigram
- Quadrigram
- Bigram

Correct
Quadrigram is not a standard term used in n-gram models. For n equals 4, the term used is four-gram.

6. Consider the sentence "The cat is on the mat". If we were to compute the probability of the sentence using a bigram model, how would we compute it? **1 / 1 point**

- P("The"|"<BOS>") * P("cat"|"The") * P("is"|"cat") * P("on"|"is") * P("the"|"on") * P("mat"|"the") * P("<EOS>"|"mat")
- P("The"|"cat") * P("cat"|"is") * P("is"|"on") * P("on"|"the") * P("the"|"cat") * P("mat"|"is")
- P("The") * P("cat") * P("is") * P("on") * P("the") * P("mat")
- P("The"|"cat") * P("cat"|"is") * P("is"|"on") * P("on"|"the") * P("the"|"mat")

Correct
Great job! This correctly represents the probability of each word given its preceding word in a bigram model.

7. What is the effect of increasing 'n' in an n-gram language model? **1 / 1 point**

- The model becomes less accurate and less efficient.
- The model captures less context but requires fewer computational resources.
- The model captures more context but requires more computational resources.
- The model becomes more accurate but less efficient.

Correct
Correct! Increasing 'n' allows the model to capture more context but at the cost of requiring more computational resources, especially memory.

8. Why might a tri-gram language model not be suitable for a small corpus of text? **1 / 1 point**

- There might not be enough examples of each possible tri-gram to accurately estimate probabilities.
- Tri-grams are too simple to capture the complexity of natural language.
- It will overfit the data, causing the model to perform poorly on unseen data.
- The computational complexity is too high for small datasets.

Correct
Great job! Tri-gram models require many examples of each possible tri-gram to accurately estimate probabilities. With a small dataset, you might not have enough examples, leading to poor model performance.

9. Why is perplexity used for intrinsic evaluation of language models? **1 / 1 point**

- It measures the model's ability to generalize to other tasks.
- Perplexity measures how well a language model predicts a sample corpus.
- It measures the model's ability to generate high fluent text.
- It measures the model's ability to correctly predict the next word in a sentence.

Correct
Correct! Perplexity measures the probability of a sequence, which directly measures how well the model represents language.

10. How are tokens that were not encountered during training often handled in language models? **1 / 1 point**

- These tokens are replaced with the nearest token using edit distance.
- These tokens are removed.
- These tokens are replaced with the nearest token alphabetically.
- They are replaced with a single special "out-of-vocabulary" (OOV) token.

Correct
Correct. OOV is often used to represent tokens that were not encountered during training.

11. Let's say you compute the perplexity on a held-out test set for two models, a bigram and a trigram language model. The bigram model has a perplexity of 45.12 and the trigram model has a perplexity of 29.64. What can you infer about these two models? **1 / 1 point**

- The trigram model is a better fit for the test data than the bigram model.
- Both models fit the test data equally well.
- The bigram model is a better fit for the test data than the trigram model.
- Nothing about the fit of the test data can be inferred from these results.

Correct
Correct: The above perplexities imply that the trigram model is a better fit.

12. To tests how good a language model was trained, your colleague suggests making up a few questions to see how well it answers them. Why is this extrinsic evaluation setup problematic? **1 / 1 point**

- A few samples is likely an insufficient amount of data to draw meaningful insights from.
- Samples that you make up might not represent a realistic application of your language model.
- The samples you make up might be in the training data.
- All of the above.

Correct
Correct! These are all problems with your colleague's evaluation plan.

13. When a context is encountered that was not seen during training, how do n-gram language models with smoothing estimate token probabilities? **1 / 1 point**

- A uniform probability distribution is applied across all tokens
- All tokens are given a probability of zero.
- All the probability is given to the "out-of-vocabulary" token.
- All the probability is given to the "end-of-sequence" token.

Correct
Correct. All tokens are considered equally likely in this case.

14. What is the best possible value for perplexity? **1 / 1 point**

- As high as possible
- The number of tokens in the sample text
- 0
- 1

Correct
Correct! A perplexity of one implies that the language model perfectly predicted the sample.

15. Why can measuring the perplexity of a model on its training data be misleading? **1 / 1 point**

- It will give an over-estimate of how well the language model generalizes to unseen data.
- It will give an under-estimate of how well the language model generalizes to unseen data.
- It will cause your perplexity to be one.
- It will cause an undefined perplexity.

Correct
Correct! Since the probabilities were generated based on the training data, you will usually have a lower perplexity on your training data than on unseen data.

16. Why is smoothing usually necessary when measuring perplexity of n-gram language models? **1 / 1 point**

- The sparsity of language means that without smoothing, many of the probabilities will be zero, resulting in undefined perplexity.
- Without smoothing, perplexity values would be negative.
- Smoothing reduces perplexity, resulting in a better model fit.
- The sparsity of language means that without smoothing, many of the probabilities will be zero, resulting in a perplexity of zero.

Correct
Correct! Not only do zero probabilities cause problems when calculating perplexity, but they do not represent the reality of natural language variation.

17. In what scenario might greedy generation be a better choice than beam search or sampling for text generation? **1 / 1 point**

- When you want to maximize the diversity of your output
- When you want to generate the least probable sequence
- When you want to generate a perfectly coherent output
- When you want to generate more probable sequences

Correct
Correct: Greedy generation will tend to generate higher-probability sequences than sampling.

18. Why might you want to apply smoothing when generating from an n-gram language model? **1 / 1 point**

- Smoothing will ensure that every context has some tokens with non-zero probability.
- Smoothing will give some probability to sequences not seen during training, improving the diversity of generation.
- Smoothing will give some probability to the end-of-sequence token in all contexts.
- All of the above

Correct
This is correct. Smoothing an n-gram language model has all of the above advantages for generation.

19. Which method among greedy generation, sampling and beam search is more likely to generate a diverse but less coherent text sequence? **1 / 1 point**

- Greedy generation
- Sampling
- Beam Search
- All of the approaches generate equally diverse and coherent sequences.

Correct
Correct! By sampling randomly from the probability distribution, the generated sequences tend to be more diverse and interesting.

20. When considering the trade-offs between different generation strategies, which of the following is a key advantage of using beam search over greedy generation and sampling? **1 / 1 point**

- Beam search is less computationally expensive.
- Beam search can generate more diverse responses.
- Beam search can generate the more probable sequence.
- Beam search is less likely to get stuck in local optima.

Correct
This is correct. Beam search will usually generate higher-probability sequences than greedy search and sampling.

21. You notice that a greedy generation over a unigram model always repeats the same token indefinitely. What's happening? **1 / 1 point**

- The "start-of-sequence" token is given the highest probability in a unigram model, so the most likely first token in a sequence is repeated.
- This is a bug in the unigram model implementation.
- The context added at each step of generation has no effect on the probabilities form a unigram model, so the same token always repeats.
- Smoothing causes the probabilities of all tokens to be identical.

Correct
Correct. The probabilities in a unigram model are identical, regardless of the context.

22. Which of these statements about generating out-of-vocabulary (OOV) tokens is true? **1 / 1 point**

- With enough Laplace smoothing, systems will always generate out-of-vocabulary tokens from n-gram language models.
- Only greedy generation can generate out-of-vocabulary tokens.
- Beam search can generate out-of-vocabulary tokens, while greedy generation and sampling cannot.
- Without smoothing, no generation strategy can generate out-of-vocabulary tokens from an n-gram language model.

Correct
Correct. Since OOV tokens are given a probability of 0 by n-gram language models.

23. What is the purpose of clipping the probability distribution to the "top-k" tokens when sampling? **1 / 1 point**

- It reduces the chance of generating very low-probability sequences
- It reduces the chance of generating very high-probability sequences
- It makes the sampling process deterministic
- It makes sampling more accurate

Correct
This is correct. Applying a "top k" filter before sampling eliminates very low-probability tokens.

24. How does the greedy generation method differ from beam search and sampling in terms of text sequence generation? **1 / 1 point**

- Greedy generation chooses the word with the highest probability at each step, while beam search maintains a set of most probable sequences, and sampling generates a diverse sequence by introducing randomness.
- Greedy generation maintains a set of most probable sequences, while beam search chooses the word with the highest probability at each step, and sampling generates a diverse sequence by introducing randomness.
- Greedy generation generates a diverse sequence by introducing randomness, while beam search maintains a set of most probable sequences, and sampling chooses the word with the highest probability at each step.
- Greedy generation and beam search are the same approach

Correct
Correct! You understand the key difference between greedy generation and beam search.